

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-07-27 | **Scope:** `shende-shweta/FSDKC` (read via GitHub) — polyglot monolith: PHP/Laravel 12 backend, Node/Express `dev-api`, React 19 + Vite + TypeScript frontend, MariaDB + MongoDB, Docker Compose

Executive Summary

EXECUTIVE SUMMARY

FSDKC ("Klearcom") is a well-structured polyglot monolith with genuinely strong agentic scaffolding: five `AGENTS.md` guides, a symmetric Discovery/Connect module layout across backend and frontend, a working GitHub Actions CI pipeline, and Docker Compose for the full stack. That foundation makes it more agent-ready than most codebases of its age. The debt is concentrated in reproducibility and quality-gate depth rather than architecture. The three most severe gaps are: (1) `backend/composer.lock` is not committed (`git/trees/main` shows `composer.json` but no lock file), so every CI run and every developer install resolves PHP dependencies afresh via `composer install` — backend builds are not reproducible; (2) **no code-style / static-analysis enforcement exists** — there is no ESLint, Prettier, or Laravel Pint anywhere, and although `phpstan/phpstan ^2.0` is declared in `backend/composer.json` there is no `phpstan.neon` and it is never invoked in `.github/workflows/ci.yml`, so agent-authored code has no automated lint gate; (3) **the database has no migration framework** — the entire schema lives in a single `docker/mariadb/init.sql` that only executes on a fresh MariaDB volume, leaving no versioned, reversible path for schema evolution. There are also self-documented debts (a deliberately reintroduced `extract()` pattern, a non-deterministic test using `random_int()`, missing service/repository layers) catalogued in `docs/CODEBASE_AUDIT_ISSUES.md`. Overall the repo is **Moderate** readiness: a real CI gate exists and can be trusted for the frontend build, but the missing lock file, absent lint layer, and flaky/shallow tests mean agent-authored output cannot yet be verified with full confidence.

Yes

TOP-LEVEL .GITIGNORE
PRESENT

1

CI/CD WORKFLOWS FOUND

19 / 18

THIRD-PARTY PACKAGES
DECLARED / WIRED

No

FRONTEND/.ENV.EXAMPLE
PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

Driven by a missing `composer.lock` (non-reproducible backend builds), no enforced lint/style gate, no DB migration framework, and a flaky/shallow test layer — no single dimension is High Risk, but every dimension carries real gaps.

Readiness Benchmark Ratings

One row per readiness dimension. "Measured" is the real state found; "Rating" is the band it falls into (worst-wins). This table is the source for the Overall Codebase Rating banner above.

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	R
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> + CI + JS lock files present; <code>composer.lock</code> missing, no CODEOWNERS/PR template, CI runs no lint step	
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	18/19 packages wired; <code>phpstan</code> declared but no config and never run; <code>dotenv</code> redundant (<code>node --env-file</code> used)	
D3	AI Tool / Agentic Readiness	ready	partial	not ready	5 <code>AGENTS.md</code> + <code>.kiro/</code> config + symmetric enumerable modules; but CI verification gate is shallow (no lint, one flaky test, <code>dev-api</code> untested)	
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	FKs with <code>ON DELETE CASCADE</code> + Mongo indexes present; no migration framework , few secondary indexes, schema+seed combined & non-idempotent	
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose + Dockerfiles + backend/dev-api <code>.env.example</code> ; no lint/formatter enforced , no <code>frontend/.env.example</code> , <code>frontend/Dockerfile</code> uses <code>npm install</code>	
D6	Automated Quality Gates & Test Determinism (additional)	lint+type+deterministic tests gate every PR	partial gate	no gate / flaky	CI runs <code>phpunit</code> + <code>tsc --noEmit</code> + <code>vite build</code> ; but no lint, no coverage, <code>ReachabilityCalculationTest</code> uses <code>random_int()</code> (flaky), <code>dev-api</code> has zero tests	

Additional readiness gaps: one additional dimension (D6 — Automated Quality Gates & Test Determinism) was observed beyond the five standard dimensions, driven by the flaky `random_int()` test (`backend/tests/Unit/ReachabilityCalculationTest.php:11-18`) and the absence of any lint/static-analysis step in CI. No further readiness gaps beyond these were observed.

8.1 Code Repository

Check	Finding	Evidence / Consequence
-------	---------	------------------------

<code>.gitignore</code> coverage	Present, adequate	<code>.gitignore</code> (top level) ignores <code>backend/vendor/</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>.env</code> , <code>dev-api/.env</code> , <code>backend/.env</code> , <code>*.log</code> . Dependency dirs, build output, and env files are all covered — low risk of committing secrets or local state.
CI/CD presence	Present but shallow	<code>.github/workflows/ci.yml</code> defines two jobs. backend : spins up MariaDB 11, sets up PHP 8.3, installs the MongoDB PECL ext, runs <code>composer install</code> then <code>vendor/bin/phpunit</code> . frontend : <code>npm ci npm install</code> then <code>npm run build</code> (<code>tsc --noEmit && vite build</code>). No lint job, no static analysis, no <code>dev-api</code> job — so lint regressions and <code>dev-api</code> breakage land unverified.
Branch protection signals	Absent	The only <code>.github/</code> content is <code>workflows/ci.yml</code> . No <code>CODEOWNERS</code> , no <code>PULL_REQUEST_TEMPLATE.md</code> , no visible required-checks config. Nothing enforces review or that CI passed before merge to <code>main</code> .
Lock files committed	Partial	<code>frontend/package-lock.json</code> (65 KB), <code>dev-api/package-lock.json</code> (54 KB), and root <code>package-lock.json</code> are committed. backend/composer.lock is missing — <code>composer install</code> with no lock resolves fresh each run, so backend dependency versions are not pinned and CI is not reproducible.

8.2 Third-Party Tools Usage

Security- / infrastructure-relevant declared dependencies and whether they are actually wired into code.

Package	Declared	Actually Wired?	Debt
<code>laravel/framework</code> ^12.0 (backend)	Y	Y	Core framework — routes, controllers, Eloquent models all use it. Current.
<code>mongodb/mongodb</code> ^2.0 (backend)	Y	Y	Wired via <code>backend/app/Services/MongoService.php</code> and <code>MongoController</code> .
<code>phpunit/phpunit</code> ^11.0 (backend, dev)	Y	Y	Invoked in CI (<code>vendor/bin/phpunit</code>) and <code>backend/phpunit.xml</code> .
<code>phpstan/phpstan</code> ^2.0 (backend, dev)	Y	N	Declared but never used — no <code>phpstan.neon</code> anywhere, no <code>phpstan analyse</code> step in CI. Pure dead dev-dependency giving a false impression of static-analysis coverage.
<code>@tanstack/react-query</code> ^5.62 (frontend)	Y	Y	Used in <code>frontend/src/hooks/useRealtimeTest.ts</code> and pages for data fetching/invalidation.
<code>react-router-dom</code> ^7.1 (frontend)	Y	Y	Routes defined inline in <code>frontend/src/App.tsx</code> .
<code>zustand</code> ^5.0 (frontend)	Y	Y	<code>frontend/src/store/uiStore.ts</code> .
<code>typescript</code> / <code>vite</code> / <code>@vitejs/plugin-react</code> (frontend, dev)	Y	Y	Build toolchain, exercised by <code>npm run build</code> in CI.
<code>express</code> ^4.21 (dev-api)	Y	Y	<code>dev-api/src/server.js</code> route handlers.
<code>cors</code> ^2.8 (dev-api)	Y	Y	Wired in the Express app.
<code>mongodb</code> ^6.12 (dev-api)	Y	Y	<code>dev-api/src/mongo.js</code> .
<code>mongodb-memory-server</code> ^10.1 (dev-api)	Y	Y	In-memory Mongo for local dev / seed.

<code>dotenv</code> ^16.4 (dev-api)	Y	Redundant	Declared, but every script runs <code>node --env-file=.env</code> (native env loading), so the <code>dotenv</code> package is not needed — dead dependency.
-------------------------------------	---	-----------	---

Net: **18 of 19** declared packages are genuinely wired. The two debts (`phpstan` unwired, `dotenv` redundant) are low-effort removals or activations.

8.3 AI Tool Usage & Agentic Readiness

Existing AI/agentic tooling — unusually mature for the repo's age:

- Five `AGENTS.md` guides: root `AGENTS.md` , `backend/app/Modules/Discovery/AGENTS.md` , `backend/app/Modules/Connect/AGENTS.md` , `frontend/src/modules/Discovery/AGENTS.md` , `frontend/src/modules/Connect/AGENTS.md` . These give an agent per-module conventions to follow.
- `.kiro/` configuration present (`.kiro/settings/mcp-bundles/`), indicating prior agent-tooling / MCP-bundle setup.
- A `.cursor/` directory exists at the workspace root (Cursor editor config), signalling prior AI-assisted-editing use.

Enumerable, isolated units of work — strong. The codebase is deliberately symmetric: Discovery and Connect exist as parallel modules on both the backend (`app/Http/Controllers/Api/{Discovery,Connect}Controller.php` , matching models) and the frontend (`pages/{Discovery,Connect}Page.tsx` , `modules/{Discovery,Connect}/`). This regular structure means an agent can enumerate "for each module, do X" tasks reliably. Better still, `docs/CODEBASE_AUDIT_ISSUES.md` already lists concrete debt with **file paths and line ranges** (e.g. `DiscoveryController.php:87-101` `buildTree()` , `LegacyDataMapper.php:12,22` `extract()`), which is effectively a ready-made agent work queue.

Honest caveat: the *verification* half of agentic readiness is weak. A safe agent loop needs a trustworthy CI gate, and here the gate has no lint/static analysis, contains a non-deterministic test, and never exercises `dev-api` . So the codebase is easy for an agent to *change* but not yet safe for an agent to *self-verify*. Some refactors are also cross-cutting — `buildTree()` is duplicated in `DiscoveryController.php` , `LegacyReportController.php` , and `dev-api/src/store.js` , and the reachability formula is duplicated 3× — so a single logical fix touches multiple files.

8.4 Database Usage

Check	Finding	Evidence / Consequence
Schema design	Partial	<code>docker/mariadb/init.sql</code> defines <code>users</code> , <code>discovery_jobs</code> , <code>discovery_nodes</code> , <code>connect_monitors</code> , <code>connect_check_results</code> . Foreign keys with <code>ON DELETE CASCADE</code> exist (<code>discovery_nodes.discovery_job_id</code> , <code>connect_check_results.connect_monitor_id</code>) and MongoDB indexes exist (<code>docker/mongodb/init.js</code> — <code>transcripts</code> and <code>test_events</code> compound indexes). But there are no secondary indexes on hot lookup columns (<code>discovery_nodes.parent_id</code> , <code>*.status</code> , <code>connect_check_results.connect_monitor_id</code> as a query key), so status/parent filters do full scans as data grows.
Migration hygiene	High-priority gap	There is no migration framework — the whole schema is one <code>docker/mariadb/init.sql</code> mounted at <code>/docker-entrypoint-initdb.d/</code> , which MariaDB runs only on a fresh data volume . Any schema change requires either a manual <code>ALTER</code> on running DBs or wiping the volume. No versioned,

		reversible path exists — a serious modernization risk. The file even comments "manual migrations per Klearcom practice," making this an explicit convention, not an oversight.
Data ownership	Reasonable	Tables are cleanly scoped by domain (<code>discovery_*</code> , <code>connect_*</code>), so per-domain extraction into services is plausible later. No single god-table shared across everything.
Seed/sample data hygiene	Gap	<code>init.sql</code> combines DDL with seed <code>INSERT</code> s (users, discovery jobs/nodes, monitors, check results) in the same file. The inserts are not idempotent (no existence guards) — they only avoid duplication because the file runs once on volume init. Sample data is synthetic (no real PII/secrets), which is good, but DDL and seed should be separated.

8.5 Development Environment

Check	Finding	Evidence / Consequence
<code>.env.example</code>	Partial	<code>backend/.env.example</code> and <code>dev-api/.env.example</code> are present. <code>frontend/.env.example</code> is missing, yet <code>docker-compose.yml</code> injects <code>VITE_API_URL</code> for the frontend — a new contributor running the frontend outside Compose has no template for that variable.
OS portability	Good	Toolchain is Docker + standard PHP/Node — no single-OS-only local dev tool. Works on Linux/macOS/Windows-with-Docker and on CI runners.
Containerization	Good, with one nit	<code>docker-compose.yml</code> orchestrates nginx, PHP app, MariaDB (with healthcheck), MongoDB, and the frontend. <code>docker/php/Dockerfile</code> and <code>frontend/Dockerfile</code> exist. Nit: <code>frontend/Dockerfile</code> runs <code>RUN npm install</code> rather than <code>npm ci</code> , so the container build ignores the committed lock file and can drift from CI.
Code style enforcement	Absent	No ESLint, Prettier, or Laravel Pint config anywhere in the tree; no pre-commit hook config; CI runs no lint step. <code>phpstan</code> is declared but unconfigured/unused (see §8.2). Configured-or-declared-but-unenforced tooling drifts immediately, and there is nothing to keep agent-authored code stylistically consistent.

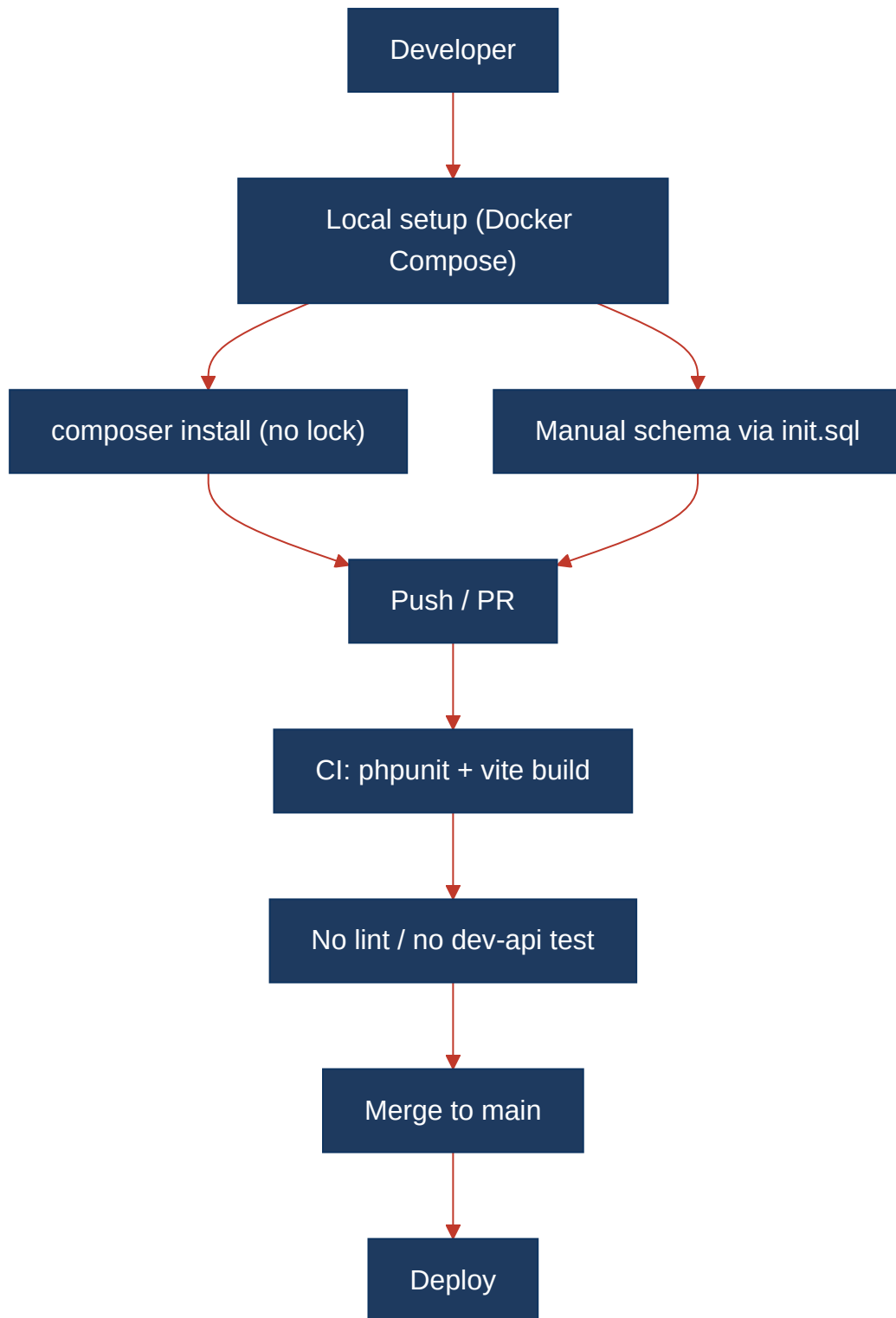
8.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Strong — symmetric Discovery/Connect modules, five <code>AGENTS.md</code> , and a fully enumerated debt list in <code>docs/CODEBASE_AUDIT_ISSUES.md</code> with files + line ranges	LOW — already listable; formalize the audit doc into tracked issues/tasks
Isolated, verifiable units of work	Good — module boundaries and file-scoped issues make changes isolatable	MEDIUM — cross-cutting duplication (<code>buildTree()</code> in 3 files, reachability formula duplicated 3×) means a single logical fix touches multiple files
CI gate to accept agent-authored output	Partial — CI runs <code>phpunit</code> + <code>tsc</code> + <code>vite build</code> on every PR, but no lint/static analysis, one flaky test, and <code>dev-api</code> untested	HIGH — harden the gate (lint, deterministic tests, <code>dev-api</code> coverage) before trusting agent merges

Repo hygiene for automation (clean checkout, no secrets)	Partial — <code>.gitignore</code> solid and no secrets committed, but <code>composer.lock</code> missing breaks reproducible backend checkout	HIGH — commit <code>composer.lock</code> ; switch <code>frontend/Dockerfile</code> to <code>npm ci</code>
Marketplace packaging readiness	Early — clean module structure and Docker packaging exist, but no versioned release/rollback strategy and no DB migration path	MEDIUM — add migrations + a release/versioning convention before packaging modules for distribution

8.7 Diagrams

Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



8.8 Actions Required

Gap	Action	Rating	Priority
<code>backend/composer.lock</code> not committed — non-reproducible backend builds	Run <code>composer update</code> and commit <code>composer.lock</code> ; keep it in sync going forward	MODERATE	CRITICAL
No code-style / static-analysis enforcement	Add ESLint + Prettier (frontend), Laravel Pint + a <code>phpstan.neon</code> (backend); wire all into CI and a pre-commit hook	MODERATE	HIGH
CI gate too shallow to accept agent output	Add a lint/static-analysis job, a <code>dev-api</code> test job, and make <code>ReachabilityCalculationTest</code> deterministic (remove <code>random_int()</code>)	MODERATE	HIGH
No database migration framework — schema only applies on a fresh volume	Adopt Laravel migrations (or a versioned SQL tool); back-fill current schema as migration <code>0001</code> ; add a rollback path	MODERATE	HIGH
<code>phpstan</code> declared but never configured or run; <code>dotenv</code> redundant	Add <code>phpstan.neon</code> + a <code>phpstan analyse</code> CI step (or remove <code>phpstan</code>); drop the redundant <code>dotenv</code> dependency	MODERATE	MEDIUM
No branch-protection signals (CODEOWNERS, PR template)	Add <code>CODEOWNERS</code> per top-level module and <code>.github/PULL_REQUEST_TEMPLATE.md</code> ; require CI checks on <code>main</code>	MODERATE	MEDIUM
Missing secondary DB indexes on FK / status / lookup columns	Add indexes on <code>discovery_nodes(discovery_job_id, parent_id)</code> , <code>connect_check_results(connect_monitor_id)</code> , and the <code>*.status</code> columns	MODERATE	MEDIUM
Cross-cutting duplication raises change cost for agents	Extract shared <code>buildTree()</code> and the reachability formula into single service utilities so one logical fix is one edit	MODERATE	MEDIUM
Schema DDL and seed data combined; container seeds non-idempotent	Split DDL from seed data; guard seed inserts behind existence checks	MODERATE	LOW
No <code>frontend/.env.example</code> ; <code>frontend/Dockerfile</code> uses <code>npm install</code> not <code>npm ci</code>	Add <code>frontend/.env.example</code> with <code>VITE_API_URL</code> ; switch the Dockerfile to <code>npm ci</code>	MODERATE	LOW

8.9 Expected Outcomes

- **Reproducible checkout:** committing `composer.lock` and switching to `npm ci` gives every developer, CI runner, and agent identical dependency trees — the baseline any automation depends on.

- **A CI gate agents can be trusted against:** adding lint + static analysis, deterministic tests, and `dev-api` coverage means a green pipeline actually implies "safe to merge," unlocking an agent-authored → CI → review → merge loop.
- **Safe, versioned schema evolution:** a real migration framework replaces the fragile fresh-volume-only `init.sql`, so schema changes become reviewable, reversible, and automatable.
- **Enforced review + consistent style:** CODEOWNERS/PR templates plus enforced formatters keep agent- and human-authored code uniform and gated, preventing drift.
- **Foundation for the Agentic Harness & Marketplace:** the already-strong module symmetry and enumerated audit backlog, combined with the hardened gate and migrations, make FSDKC a credible target for systematic agent-driven refactoring and eventual module packaging.